.NET Multi-platform App UI

Богданова Арина

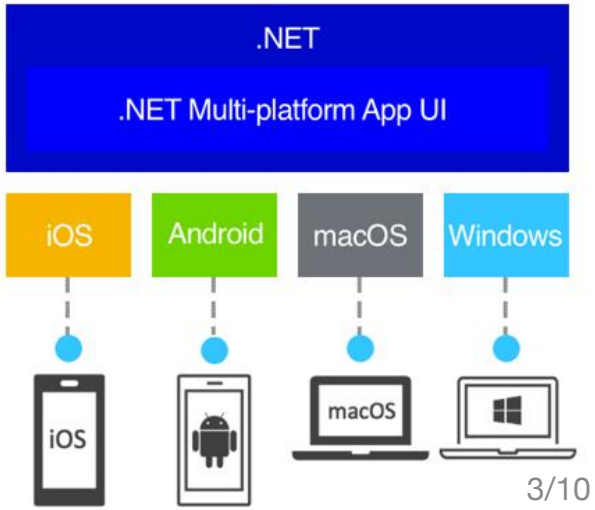
02.12.2025

Что такое .NET MAUI?

- Кроссплатформенная платформа для создания мобильных и классических приложений с помощью C# и XAML
- .NET MAUI — часть современного .NET
- Является логическим продолжением Xamarin.Forms
- Полностью интегрирован в экосистему .NET 6+
- Поддерживается как официальная технология Microsoft для кроссплатформенной разработки нативных приложений
- Отлично интегрирован в Visual Studio

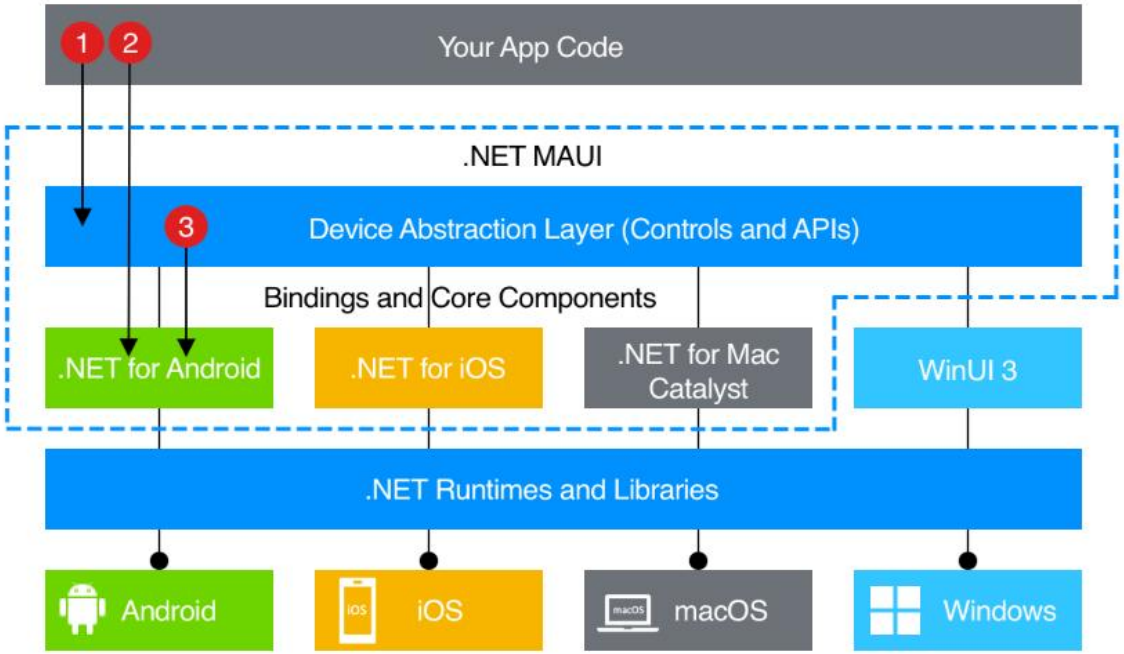
Кроссплатформенность

- “Write Once, Run Anywhere”
- Общая бизнес-логика и UI-слой
- Платформы: .NET для Android, .NET для iOS, .NET для Mac Catalyst и библиотеки Пользовательского интерфейса Windows 3 (WinUI 3)
- Поддержка single-project approach: один проект вместо четырёх



Как работает?

- BCL абстрагирует сведения о базовой платформе от кода.
- Среда реализуется с помощью:
  - MONO для Android, iOS и macOS
    - iOS: обязательная Ahead-of-Time (AOT) компиляция в ARM-инструкции
    - Android: использует JIT, но с возможностью AOT-компиляции для критичных участков
    - macOS: JIT через .NET runtime
  - CLR из .NET Core для Windows
- Платформенные различия обрабатываются через условную компиляцию и platform-specific API
  - Условная компиляция (Conditional Compilation) - механизм C#, при котором можно включать или исключать участки кода в зависимости от того, под какую ОС собирается приложение.
  - Platform-specific API - специальные методы или свойства, которые работают только на одной платформе, но вызываются общим способом.



Базовая класс-библиотека (BCL) и нативные API

- BCL абстрагирует платформу от кода
  - Использует:
    - Mono (рантайм для iOS/Android, с AOT-компиляцией)
    - .NET Core (для Windows/macOS)
- Автоматическое переключение на нативные API (NS, UIKit, WinUI и др.)

UI в .NET MAUI

- xaml-подобная технология (интерфейс описывается декларативно в формате xml) в отличие от Windows.Forms, где подход был императивным
- Сохранена возможность динамического создания элементов в C#

csharp

```csharp
var button = new Button();
button.Text = "Нажми меня";
button.Click += OnButtonClicked;
button.BackColor = Color.LightGray;
button.Font = new Font("Segoe UI", 18);
button.Location = new Point(20, 20);
button.Size = new Size(150, 40);
this.Controls.Add(button);
```

xml

```xml
<Button Text="Нажми меня"
        Clicked="OnButtonClicked"
        BackgroundColor="LightGray"
        FontSize="18"
        HorizontalOptions="Center"
        Margin="20" />
```

csharp

```csharp
// Код должен располагаться в C#-файле code-behind, например, в MainPage.xaml.cs
private void OnButtonClicked(object sender, EventArgs e)
{
    Console.WriteLine("Кнопка нажата!");
}
```

MAUI Graphics и Essentials API

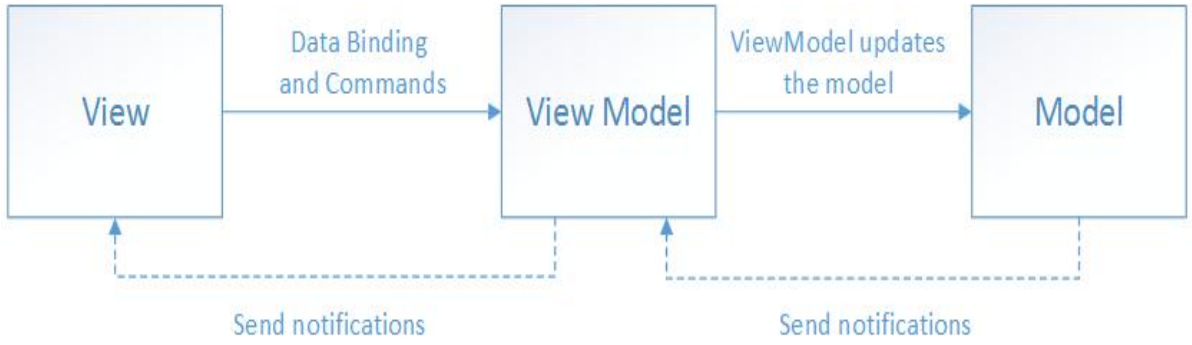
- MAUI.Graphics — низкоуровневый API для отрисовки кастомной графики (Canvas, SkiaSharp-подобные возможности)
- Essentials API предоставляет унифицированный доступ к:
  - Геолокации
  - Акселерометру
  - Файловой системе
  - Камере
  - Батарее
  - Clipboard и многому другому

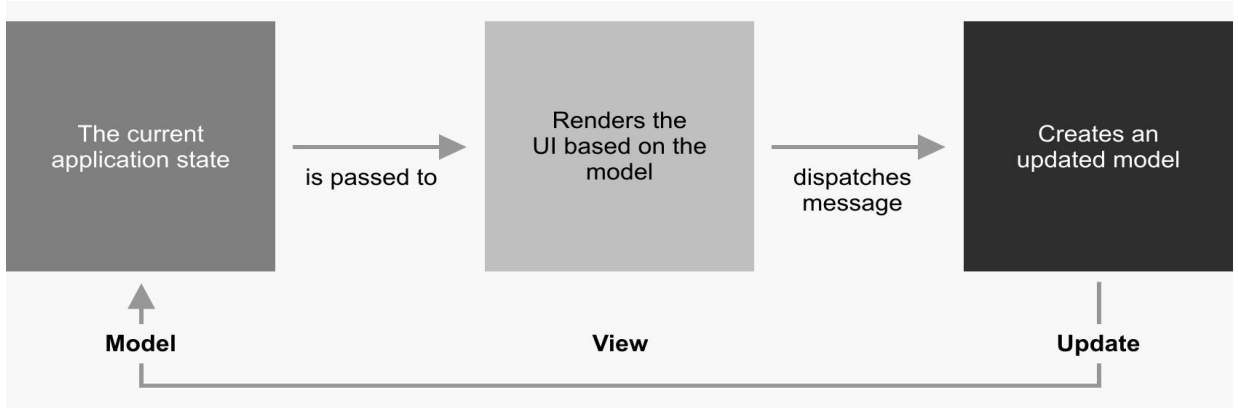
Встроенный Dependency Injection

- MAUI построен на встроенном DI-контейнере .NET
- Упрощает внедрение сервисов и тестирование
- Пример регистрации:
  - builder.Services.AddSingleton<IMyService, MyService>();
- Доступ из IServiceProvider через Application.Current.Services

Архитектурные паттерны: MVVM и MVU

- MVVM (Model–View–ViewModel) — основной паттерн:
  - Model — бизнес-логика приложения и работа с данными,
  - View — XAML-интерфейс
  - ViewModel — логика, привязка к View через BindingContext
- Поддержка MVU (Model–View–Update) — альтернатива для функционального стиля (в основном в экспериментах и Community Toolkit)





Blazor в MAUI — гибридный подход

- Blazor Hybrid позволяет запускать Blazor-компоненты внутри нативного MAUI-приложения
- UI создаётся с помощью Razor-синтаксиса
- Используется WebView, но с нативной производительностью и доступом к платформе через JSInterop → C#